

DS420 Applied Machine Learning - Final Report

Vincent Buchner, Soren Basnet, Victor Azih | Dec 22nd, 2024 | Dr. Thuy Do

Checklist

- ☒ Investigate dataset's for ensemble learning and DNNs.
- ☒ State the clearly desired performance.
- ☒ Select diverse algorithms for ensemble learning.
 - ☒ Bagging
 - ☒ Voting
 - ☒ Gradient
 - ☒ Ada
 - ☒ Random Forest
 - ☒ Stacking
- ☒ Assign different ensemble algorithms to team members.
 - ☒ Bagging, Random Forest - Victor
 - ☒ Voting, Gradient - Vincent
 - ☒ Ada, Stacking - Soren
- ☒ Develop ensemble learning models.
- ☒ Annotate ensemble learning code with comments (e.g., #prepare data, #train model).
- ☒ Assign different DNN architectures to team members.
 - ☒ Critical decision making and exploration - Vincent and Soren
 - ☒ DNN fine tuning with dropout and regularization (for both transformed and untransformed Response Variable) - Vincent
 - ☒ DNN fine tuning with dropout (for both transformed and untransformed Response Variable) - Soren
 - ☒ DNN with Grid Search - Victor
- ☒ Train and fine-tune DNN models.
- ☒ Interpret DNN models and document insights.
- ☒ Compare two algorithms from Project #1. With ensemble learning and DNN methods from Project #2. Highlight performance improvements or explain lack thereof.
- ☒ Include team member contributions at the start of each code file.

Abstract

This project aims to improve upon the performance of previous models by using ensemble learning and deep neural networks (DNN). While the best model from the midterm, SGDRegressor, achieved an MAE of 214.78, RMSE of 279.75, and R^2 of 0.72, we aim to reduce these metrics using more complex algorithms. Various models, including Voting Regressor, Gradient Boosting, AdaBoost, and DNN, were tested, but DNN with dropout and regularization showed comparable error rates. Transforming the response variable with the Yeo-Johnson method led to improved MAE (~ 29) and RMSE (~ 38) for DNN, but R^2 remained low ($\sim 0.05\%$), indicating that the model captured limited variance in the data. This suggests that while DNNs reduce error, they still struggle to explain the underlying variability.

Responsibilities :

- Victor Azih : Run Ensemble and DNN analysis
- Vincent Buchner : Run Ensemble and DNN analysis along with fine tuning hyperparameters
- Soren Basnet : Run Ensemble and DNN analysis along with fine tuning hyperparameters

Desired Performance

Since our Kaggle dataset lacks a human metric for the predicted 'y-value,' we aimed to surpass the performance of our models from the midterm project. In that project, our best linear model was the SGDRegressor, achieving metrics of MAE = 214.78, RMSE = 279.75, and $R^2 = 0.72$. Our goal is to improve upon these results by using a variety of ensemble learning and deep learning methods to reduce both RMSE and MAE. We plan to leverage more complex algorithms, focusing on model diversity and training techniques. Ultimately, we hope this will allow us to better capture the patterns within the data and improve model generalization.

Models

Voting Regressor - Vincent

This notebook focuses on building a predictive model using ensemble methods, specifically the VotingRegressor. There was no need for grid search of this model, as the parameters for this model rely on the estimators. Therefore, we just used our best models from Project #1 as our estimators for this model that are already hypertuned from the midterm project. Our final result from this model was RMSE = 278.23, MAE = 213.19, and $R^2 = 0.03$.

Gradient Boosting Regressor - Vincent

This notebook demonstrates the use of a Gradient Boosting Regressor, a machine learning model that optimizes predictions by sequentially combining weaker models to reduce error. We fine-tuned key hyperparameters, including learning_rate, n_estimators, min_samples_split, min_samples_leaf, min_weight_fraction_leaf, min_impurity_decrease, max_features, and others (details available in the notebook). The final model achieved performance metrics of MAE = 211.26, RMSE = 275.33, and $R^2 = 0.053$.

AdaBoost Regressor - Soren

This notebook explores the AdaBoostRegressor, a machine learning model that improves predictive performance by iteratively combining weak learners. Key hyperparameters such as `n_estimators`, `estimator`, `learning_rate`, and `loss` were fine-tuned, with default estimators used to maintain consistency across models. The grid search yielded results of MAE = 345.97, RMSE = 459.39, and $R^2 = -1.636$, while the default model significantly outperformed it with MAE = 236.64, RMSE = 292.43, and $R^2 = 0.292$. The default model performed better because over-tuning during grid search likely led to overfitting or ineffective parameter combinations.

Stacking Regressor - Soren

This notebook utilizes the Stacking Regressor, an ensemble method that combines predictions from multiple base models to improve overall performance by leveraging their strengths. The hyperparameters tuned include `passthrough`, `final_estimator`, and `cv`, while the base estimators (Decision Tree, Ridge, and SGD) were configured using the best hyperparameters from Project #1. The final model achieved MAE = 208.52, RMSE = 282.30, and $R^2 = 0.004$.

Random Forest Regressor - Victor

This notebook uses the Random Forest Regressor, an ensemble learning method that builds multiple decision trees during training and averages their predictions to improve accuracy and reduce overfitting. Hyperparameters such as `n_estimators`, `max_depth`, and `min_samples_split` were fine-tuned through grid search (details available in the notebook). The model achieved final metrics of MAE = 211.14, RMSE = 275.37, and $R^2 = 0.05$.

Bagging Regressor - Victor

This notebook leverages the Bagging Regressor, an ensemble learning technique that trains multiple base models on random subsets of data to reduce variance and improve stability. The model was fine-tuned using hyperparameters such as `n_estimators`, `max_samples`, `max_features`, `bootstrap`, `bootstrap_features`, and a `base_estimator` set to a DecisionTreeRegressor with a maximum depth of 8. After tuning, the model achieved results with metrics of MAE = 211.20, RMSE = 275.11, and $R^2 = 0.05$.

The Gradient Boosting Regressor performed the best with MAE = 211.26, RMSE = 275.33, and $R^2 = 0.053$, demonstrating strong predictive power through careful hyperparameter tuning and sequential model improvement. The AdaBoost Regressor performed the worst, with grid search results yielding MAE = 345.97, RMSE = 459.39, and $R^2 = -1.636$, likely due to overfitting or ineffective parameter tuning. The Stacking Regressor, with base models tuned from Project #1, achieved MAE = 208.52, RMSE = 282.30, and $R^2 = 0.004$, having the best MAE out of all the ensemble models.

Deep Neural Network

Goal : Given the complexity of the model, we anticipate achieving results superior to those of the previously discussed models in this paper and in Project 1. Our evaluation metrics include RMSE, MAE, and R-squared, and we aim for optimal results from the aforementioned measures.

Basic Neural Network :

This part of the project was spearheaded by Victor with grid search. The result of the model was not optimal and therefore the team decided to opt for a more complex Deep Neural Network.

DNN with Dropout Rate (with Random Search) :

Soren was tasked with running the analysis using Random search which included tuning dropout rate. The result derived from the analysis was not as expected :

MAE: ~215.66

RMSE: ~279.89

R-squared: ~2.1%

The low R-squared value is of concern as the R-squared value indicates that the model is only explaining 2.1% of the variation in the data.

DNN with Dropout Rate and Regularization (with Random Search)

Vincent was then assigned the task of running the Random Search which included tuning both regularization and dropout. Optimal parameters were :

The result derived from the analysis was as follows :

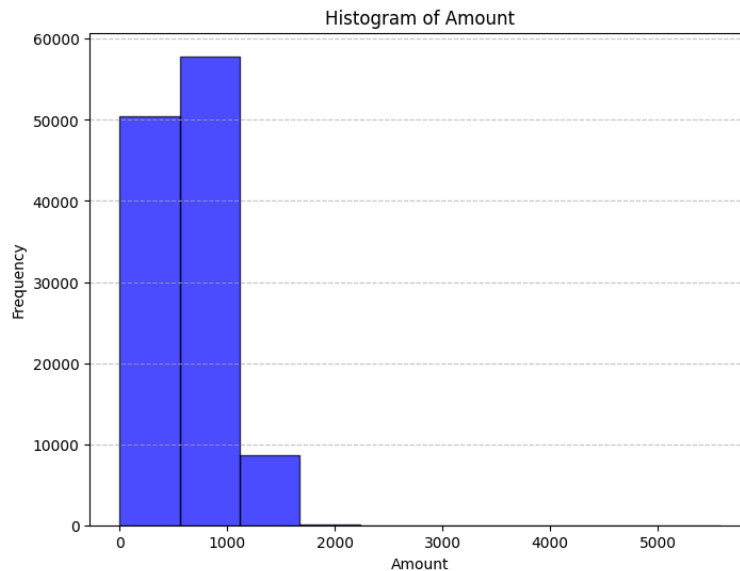
MAE : ~ 209.9

RMSE : ~ 274

R-squared : 0.06 %

Despite these efforts, our DNN model did not achieve the desired error rates. The MAE and RMSE values remained comparable to those of less complex models. With a DNN, we expected the model to capture nuanced patterns and accurately reflect trends with minimal error, but this outcome fell short of our expectations.

Response Variable Exploration - Yeo-Johnson Transformation (Vincent and Soren)



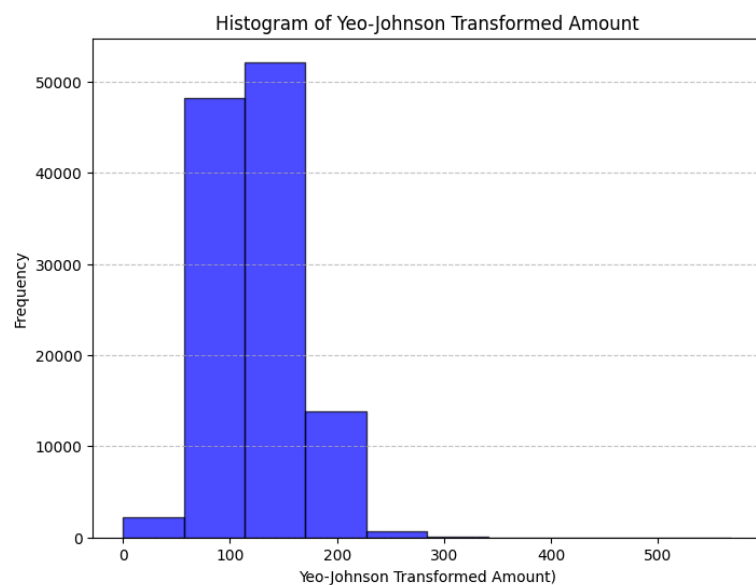
This task was experimented by **Soren and Vincent** where transformation of the Response Variable was done using Yeo-Johnson Transformation. The underlying pattern of the Response variable (Amount) was negatively skewed which is a disadvantage to the DNN.

Therefore, exploration with transformation of the data was necessary. Since the response variable contained 0 or negative value for the dataset we are working with, we opted for Yeo-Johnson Transform. (Machine Learning Mastery).

The response variable's histogram looked something like this :

DNN for Transformed Response

This analysis was conducted by Soren with the team's approval. Transforming the response



variable yielded the following results:

- MAE: 29.09
- RMSE: 38.25
- R^2 : 0.055

The MAE and RMSE values were the most optimal observed so far in the dataset analysis. However, the R-squared value remained low, indicating that the model explained only a small fraction of the variation in the data.

DNN with Random Search (Dropout and Regularization):

Vincent took over the analysis from Soren, as the team saw potential improvements in the R-squared value as well as further refinement of MAE and RMSE. The results were as follows:

- MAE: 29.2
- RMSE: 38.56
- R^2 : 0.04

Compared to the previous analysis, these metrics showed a marginal decline in performance. The addition of regularization did not improve R-squared and resulted in slightly worse MAE and RMSE values, suggesting limited benefits for this approach.

Table Summary of the metrics :

Model	MAE	RMSE	R-Squared
DNN (w/ Dropout)	215.6626196	279.8847146	0.02150708748
DNN (w/ Dropout & Regularization)	209.8949304	274.3141469	0.06006951872
DNN (w/ Transformation, Dropout)	29.09461891	38.24910367	0.05523486427
DNN (w/ Transformation, Dropout & Regularization)	29.24275652234533	38.55726137414101	0.03995035225

Here, we observe that the MAE of the DNN models using transformed data outperforms the models without transformation. An MAE of approximately 29 indicates that, on average, the model's predictions differ from the actual values of the response variable by just 29 Indian Rupees. This amount is minimal even in the Indian context, and when converted to USD, it becomes even less significant. Therefore, we can conclude that the MAE of the DNN model is satisfactory.

An RMSE of approximately 38 for the DNN models with transformation indicates that, on average, the model's predictions deviate from actual values by 38 Indian Rupees.

However, a significant issue arises with the R-squared value, which fails to yield an optimal result. A low R-squared value suggests a poor fit for the model, potentially due to noisy data or high variability in the response variable. It may also indicate that the features used have weak predictive power.

Despite this, the poor R-squared value cannot be taken as a definitive explanation for the DNN model's underperformance, as other earlier models have demonstrated their ability to explain a higher proportion of variance in the response variable, with R-squared values closer to 1.

Conclusion: Table summary for the Project 2 including all analysis :

Model	MAE	RMSE	R ²
Voting Regressor	213.19	278.23	0.03
Gradient Boosting Regressor	211.26	275.33	0.053
AdaBoost Regressor (Grid Search)	345.97	459.39	-1.636
AdaBoost Regressor (Default)	236.64	292.43	0.292
Stacking Regressor	208.52	282.3	0.004
Random Forest Regressor	211.14	274.81	0.06
Bagging Regressor	212.93	276.35	0.05
DNN (w/ Dropout)	215.66	279.89	0.0215
DNN (w/ Dropout & Regularization)	209.9	274	0.0601
DNN (w/ Transformation, Dropout)	29.09	38.25	0.0552
DNN (w/ Transformation, Dropout & Regularization)	29.2	38.56	0.04

While the DNN models improved the MAE and RMSE compared to simpler models, they still failed to capture a significant amount of variance in the data, as indicated by the low R-squared values. This suggests that while the models can make accurate predictions, they struggle to fully explain the underlying patterns in the data.

The AdaBoostRegressor (Grid Search) performed the worst with extremely high MAE (345.97) and RMSE (459.39), as well as a negative R-squared, indicating severe overfitting or poor generalization. The AdaBoostRegressor (Default) performed better but still lagged behind the DNN, with an MAE of 236.64, RMSE of 292.43, and an R-squared of 0.292. AdaBoostRegressor (Default), Stacking Regressor, Random Forest Regressor, and Bagging Regressor have relatively better MAE and RMSE, but the R-squared values are still low.

Future steps include exploring additional feature engineering to improve model performance, experimenting with more advanced techniques such as hyperparameter tuning and transfer learning, and considering alternative models like XGBoost

Additionally, further investigation into data transformations and addressing the high variability or noise in the response variable may help enhance model generalization and predictive power with ensemble models. Due to time constraints, we were not able to experiment with all the models with transformed Response Variables.

From Project 1 :

Model	MAE	RMSE	R-Squared
Linear Regression	214.82	279.76	0.72
Ridge	214.82	279.76	0.72
SGDRegressor	214.8	279.97	0.72
K-Neighbors Regressor	233.77	304.02	0.67
Decision Tree Regressor	227.22	297.18	0.68
Random Forest Regressor	214.57	280.3	0.72

From the result from Project 1, we can see that linear models like Linear Regression, Ridge and SGD Regressor out performs the DNN and ensemble learning methods in terms of R-squared value (0.72).

Digging deeper, we see that AdaBoostRegressor (Grid Search) shows a very poor performance with a negative R-squared value. Compare that with the aforementioned linear models, we see that the R-squared value is close to 1 (0.72). Random Forest Regressor in Project 2 performs better than linear models in terms of MAE and RMSE as the MAE value for RF in Project 2 is 211.14, RMSE is 274.81 but R-squared value is 0.06 (which linear models out performs | Refer to table above)

While this paper will not conclude with a single optimal model, our team will recognize that DNN models with transformation show promising error reduction but fail to capture significant portions of the variance in the data - which makes model interpretation less effective.